Cluster Computing (2025) 28:933
https://doi.org/10.1007/s10586-025-05584-7

Optimizing machine learning operations in multi-cloud infrastructure: a framework for unified deployment management and topology discovery

Hao Wei[1] · Xabiel García Pañeda[1] · Joaquín Salvachúa Rodriguez[2]

Received: 22 November 2024 / Revised: 14 May 2025 / Accepted: 5 June 2025 / Published online: 29 September 2025
© The Author(s) 2025

Abstract
Machine learning (ML) transitioned from a purely academic discipline to an applied field, gaining strategic importance in various industries. Meanwhile, Machine Learning Operations (MLOps) has been widely adopted by enterprises as a comprehensive approach for developing and managing machine learning applications. Despite its advantages, challenges remain. The rising demand for flexibility and scalability has led organizations to embrace multi-cloud and hybrid cloud architectures as preferred solutions. However, the autonomous and distributed nature of modern application development, combined with the complexity of training and deploying machine learning models, makes unified operational management impractical, and this will further affect application quality and efficiency. To address these challenges, this paper proposes a framework to manage model training and deployment in a multi-cloud environment. This framework uses a policy-based resource provisioning approach, agent-based application topology reconstruction, and a visualization dashboard. It aims to provide a cloud provider-neutral solution that enhances the quality of application operations. The framework design is introduced, followed by the implementation of a proof-of-concept prototype. Experiments conducted in various empirical scenarios demonstrate that the proposed framework effectively manages deployment resources while providing clear visibility and control across multiple clouds. The results confirm that this framework enhances control over deployment resources and optimizes model deployment efficiency in multi-cloud infrastructure.

Keywords MLOps · DevOps · Cloud computing · Policy-base control

Mathematics Subject Classification 68U03 · 68U35

1 Introduction

Machine learning has progressed from a theoretical concept in academia to a transformative technology in industry. Its capabilities have expanded significantly with advancements in computing power and data availability. In academia, researchers have explored fundamental problems like supervised learning, unsupervised learning, and reinforcement learning [1]. These theoretical underpinnings have paved the way for ML applications that are now integrated to business operations. Today, ML is embedded in diverse sectors such as healthcare, finance, retail, and transportation, offering predictive analytics, anomaly detection, and personalized recommendations [2]. This widespread adoption has brought about unique engineering challenges due to the need for scalable, reliable, and reproducible ML systems.

✉ Hao Wei
uo302709@uniovi.es

Xabiel García Pañeda
xabiel@uniovi.es

Joaquín Salvachúa Rodriguez
joaquin.salvachua@upm.es

1 Departmento de Informáctica, Universidad de Oviedo, Calle Pedro Puig Adam, Gijón 33203, Spain

2 Departamento de Ingeniería de Sistemas Telemáticos, Universidad Politécnica de Madrid, Avenida Complutense, Madrid 28040, Spain

Consequently, the application of machine learning increasingly faces engineering problems, including data management, model deployment, and the monitoring of large-scale, real-world systems [3].

Machine learning operations is an emerging discipline that combines the principles of DevOps and data engineering to manage the end-to-end lifecycle of machine learning models [4]. MLOps encompasses practices, tools, and frameworks designed to streamline the deployment, monitoring, and governance of ML models [4]. At its core, MLOps automates the continuous integration and continuous delivery (CI/CD) pipeline for ML, ensuring that models are reliably deployed while maintaining consistency, security, and compliance [5]. MLOps integrates data management, model training, model serving, and performance monitoring into a cohesive workflow, allowing data scientists and ML engineers to collaborate effectively. By reducing the friction between model development and deployment, MLOps accelerates the time-to-market for ML solutions and enables organizations to scale their applications efficiently.

Unlike traditional software applications, machine learning (ML) applications present unique deployment challenges. These applications often require managing and processing large datasets, adding complexity to the deployment process. Furthermore, ML applications involve two distinct phases: model training and model serving, each demanding different infrastructure requirements and operational considerations. This inherent two-phased nature introduces complexities not typically found in traditional application deployments. Additionally, ML models often undergo continuous model iteration, requiring frequent retraining and updating as new data becomes available. The deployment pipeline must be flexible to seamlessly integrate updated models into the production environment.

MLOps plays a critical role in businesses by ensuring that machine learning models deliver consistent value in production environments. It facilitates collaboration between data scientists, ML engineers, and operations teams, leading to improved productivity and innovation. With automated model deployment and monitoring, MLOps enables rapid experimentation and iteration, allowing businesses to quickly adapt to changing market demands.

The rising demand for flexibility and scalability has led organizations to embrace multi-cloud and hybrid cloud architectures as preferred solutions [6]. Multi-cloud infrastructure allows enterprises to leverage the best features of various cloud service providers while avoiding vendor lock-in. Organizations can distribute workloads across multiple cloud infrastructures to optimize costs, ensure high availability, and enhance disaster recovery capabilities. In hybrid cloud setups, businesses integrate on-premises infrastructure with public clouds, providing the benefits of both private and public computing environments. This approach not only addresses the unique compliance and data security requirements of various industries but also enables rapid scaling to accommodate fluctuating workloads. The strategic use of multi-cloud infrastructure also aligns with the increasing adoption of containerization and microservices architecture, further enhancing the flexibility of enterprise applications [7].

As enterprises benefit from the flexibility and scalability offered by multi-cloud infrastructures, understanding the important role of resource provisioning in MLOps becomes critical. Effective resource provisioning is essential because it directly impacts the efficiency, performance, and cost-effectiveness of ML applications. In multi-cloud infrastructure, where resources can vary significantly between platforms, ensuring that the right resources are available at the right time for training and deployment is crucial. As we transition into ML application engineering, it's important to investigate into the specific operational challenges faced by MLOps, particularly the task of resource provisioning in such heterogeneous infrastructure.

Unlike traditional software applications, machine learning applications present unique deployment challenges. Model training is computationally intensive, often requiring specialized hardware like GPUs or TPUs, and needs frequent retraining as new data becomes available. This creates dynamic resource demands, unlike traditional applications. In contrast, serving involves deploying trained models for real-time inference, where low latency is crucial. Optimizing resources for both high-throughput training and low-latency serving requires a tailored approach, as traditional applications do not experience such bifurcation in their deployment. ML applications also require continuous model iteration, where models are regularly retrained and updated. This creates added complexity in deployment, as the pipeline must handle both training in high-performance environments and serving in low-latency environments. This frequent updating and iteration is a key difference from general applications, which typically do not require such ongoing changes post-deployment.

Different cloud providers offer varying pricing models, infrastructure, and resource types, complicating resource provisioning [8]. MLOps teams must ensure optimal resource allocation while balancing cost and performance. This challenge is particularly critical in training, where large datasets require significant computational power, and in deployment, where latency and availability are important.

The computational cost of training and inference, particularly for large-scale models, can be substantial across multiple cloud infrastructure. Furthermore, energy consumption in data centers raises concerns about the environmental impact, as energy-intensive computations contribute

significantly to the carbon footprint. Balancing computational demands with sustainability goals is a pressing issue.

Selecting the appropriate hardware accelerators, such as GPUs, TPUs, or FPGAs, requires careful consideration of compatibility, performance, and cost across cloud platforms. Some clouds may offer specialized hardware that is unavailable elsewhere, necessitating strategic decisions about where to train and deploy models.

Delivering machine learning models to end-users across different regions and platforms requires optimized deployment strategies to reduce latency and improve user experience. In multi-cloud infrastructure, this often means deploying models closer to end-users geographically.

Maintaining consistent data governance practices across multiple cloud infrastructure is challenging. Organizations must implement robust governance frameworks to ensure data quality and security. Cross-cloud data management becomes a significant hurdle.

From an organizational perspective, the separation between data science and operations teams presents a significant challenge. Data scientists focus primarily on model development, while operations engineers handle deployment and monitoring. This division often leads to communication gaps, conflicting priorities, and difficulties in transitioning models from experimentation to production. Aligning these teams through cross-functional collaboration and standardized workflows is crucial for effective MLOps.

To address these challenges, this paper proposes a framework that utilizes a policy-based control approach, employs agent-based application deployment topology reconstruction, and offers a visualization dashboard. This framework, focused on improving the deployment management of machine learning models during the training and serving phases within a multi-cloud environment, aims to provide a viable and cloud provider-neutral solution that improves the quality of application operations. It is designed to seamlessly integrate with other existing MLOps tools and frameworks, such as Kubeflow or MLflow, that handle broader lifecycle tasks, allowing specialized systems to manage different MLOps lifecycle components. This ensures that deployments are optimized and policy-compliant without replacing or covering the entire MLOps lifecycle.

We have developed a proof of concept prototype to gain deeper insights into the proposed framework. A series of experiments were conducted, and the results from these experiments demonstrate the effectiveness of the framework.

The expected contributions of the paper are:

1. A novel policy-based method for regulating training and serving deployment resource in enterprise multi-cloud environment;
2. A cloud platform agnostic application deployment topology discovery and a dashboard to visualize the topology of deployment;
3. A prototype implementation that is evaluated using a real world application in different scenarios.

We believe that our research could be a valuable aid for ML application operation management in cloud.

The subsequent sections of this paper are structured as follows: Sect. 2 outlines the foundational aspects of our study and discusses the motivation of our proposal. In Sect. 3, we explore existing research related to the field of multi-cloud resource selection and management, application topology discovery. A high-level overview of the proposed framework and its major components are discussed in Sect. 4. Section 5 introduces a proof of concept prototype alongside a series of experiments. Section 6 is devoted to the discussion of experimentation. Finally, Sect. 7 concludes the paper and envisages the future work.

2 Motivation and background

2.1 Heterogeneous infrastructure

In the dynamic landscape of the modern enterprise, subdivided IT infrastructures are increasingly being adopted by companies to address complex business and technological requirements. Strategic segmentation, often seen in multi-cloud infrastructures across multinational organizations, has become prominent to optimize management, reduce business risks, improve efficiency, and ensure compliance with legal standards. Moreover, using a multi-cloud infrastructure can mitigate the variations in pricing and performance among different cloud providers, allowing organizations to select the best combination of services that align with their specific needs [9].

The decision to break large IT infrastructure into smaller, more manageable domains reflects the need for domain-specific configurations that are both cost-effective and operationally efficient. For instance, spreading resources across three geographical data centers and subdividing them based on unique security, service-level requirements makes large IT infrastructures more manageable. This approach allows for customized data recovery and security policies, ensuring that each segment operates under optimal, domain-specific conditions.

Furthermore, this multi-domain strategy aligns with broader organizational objectives, such as reducing computation costs, lowering environmental impacts, and adhering to legal requirements. Each segmented domain is optimized not only for performance but also for cost-effectiveness and

sustainability. In addition, hardware is strategically selected within each domain to meet specific performance needs, boosting overall system efficiency while minimizing costs.

Thus, the drive to divide IT infrastructures in modern enterprises is based on the need to create a more controlled, efficient environment. This approach effectively addresses current business and technological demands in a rapidly changing world. However, this segmentation also introduces new challenges in application deployment within these heterogeneous infrastructure. Traditionally, aligning the proper domain with deployment activities has been a complex and time-consuming process, particularly given the large number of applications and packages to be deployed. This complexity requires innovative solutions to streamline deployment processes and ensure optimal resource utilization.

2.2 Diversified organization setup

The model-driven ML project is transitioning to a product-oriented engineering approach. This brings about a significant shift in both mindset and project culture [10]. Traditionally, ML projects focus primarily on creating models, and they were often measured in accuracy and performance. However, as we begin to integrate these models into real-world applications, the focus expands to the entire lifecycle of a product, from design to deployment and monitoring [11]. This engineering approach ensures that ML projects are scientifically sound, practical, and reliable. It needs to apply software engineering principles to ML projects, in order to make ML applications scalable, robust, and efficient.

In common practice, a ML project typically has at least two distinct teams with different specializations, data scientist team and engineer team. First, data scientists concentrate on data analysis and model training; in other words, they develop models to learn effectively from dataset. Second, engineers team specializes in managing resources and maintaining applications throughout continuously deployment, comprehensive observability [12]. In some common cases, one engineer team even needs to work with several data scientist teams. Since these are two distinct specializations and not often an one-to-one collaboration, teams often work in isolation. Additionally, modern software technologies and practices, such as microservice architecture, agile methods, empower self-management and autonomy within software organizations [13]. This lack of coordination makes it difficult to integrate ML models into production-level applications. Furthermore, there is a notable shortage of professionals skilled in both machine learning and DevOps engineering. To address these challenges, we need to establish a more integrated and automated approach while respecting the autonomous team culture.

3 Related works

This section gives a brief overview of existing research related to the field of multi-cloud resource selection and management and application topology discovery.

3.1 Multi-cloud resource management in MLOps

While there has been considerable research focused on resource management in multi-cloud infrastructure, much of the existing work addresses specific aspects such as task scheduling, GPU optimization, or job distribution. However, many of these approaches still lack a comprehensive solution that integrates policy-driven control and compliance management, particularly in the context of MLOps. In this section, we review existing frameworks and tools that have contributed to the multi-cloud domain, highlighting both their strengths and their limitations when compared to the proposed framework.

Schridhar et al. [14] introduces a heuristic-based approach to optimize resource allocation for cloud-native machine learning workloads, particularly focusing on efficiently distributing the training of large models across multiple machines. It splits efficiently a machine learning model's computational graph across multiple GPUs to optimize both latency and resource usage. However, this research focuses more on distributing machine learning workloads across multiple machines rather than optimizing resource provisioning.

Chiang et al. [15, 16] introduce DynamoML and HYPREL. DynamoML is a Kubernetes-based[1] platform that can schedule jobs by sharing GPU usage, grouping training jobs and auto scale inference server instances. HYPREL is a topology-aware scheduling algorithm, it optimizes task placement by using hypergraph min-cut partitioning to model and reduce task dependencies. Their approach improves GPU utilization and minimizes communication overhead. However, this research assumes that infrastucture is one complete piece under management of Kubernetes. In reality it often not true. In multi-domain infrastructure, this approach can provide little help.

Pangea [17] is an MLOps tool designed to automate the creation of infrastructure and deployment of analytic pipelines in various environments. It simplifies the process by automatically generating infrastructure, configuring software, and deploying ML models. It supports a wide range of deployment contexts and provides a user-friendly web client. Their approach to automatically generating pipelines is an innovative way to facilitate the effort to implement pipeline with diversified infrastructure and technologies.

[1] https://kubernetes.io/.

This tool can be an ideal interface to isolate ML scientist from heavy infrastructure coding tasks. Unfortunately, there is still considerable uncertainty with regard to the feasibility to maintain such tool with large amount of supported infrastructure and technologies, especially in this fast growing field.

Zhang et al. [18] introduce Sinan, a system that uses machine learning to manage resources in cloud microservices. Sinan dynamically adjusts resource allocations to meet QoS targets, employing Convolutional Neural Networks and Boosted Trees for accurate performance predictions. Evaluations show that Sinan improves resource efficiency and scalability compared to traditional methods, effectively handling the complexities of microservices dependencies. This approach highlights the effectiveness of ML in optimizing cloud resource management. The main limitation is that QoS is not only factor we need to consider in resource management of cloud infrastructure.

Gholipour et al. [19] propose an innovative energy-aware resource management technique that integrates virtual machine (VM) and container consolidation to enhance energy efficiency in cloud data centers. Their approach uses a multi-criteria decision-making policy to optimize resource utilization through the joint migration of VMs and containers.

Zhu et al. [20] presents a novel task scheduling framework for multi-cloud computing, addressing the need to satisfy both performance and trust QoS constraints. It proposes a matching and multi-round allocation (MMA) algorithm that optimizes makespan and total cost while considering security and reliability constraints, a combination not fully addressed in previous research. Another task scheduling algorithm [21] is introduced, it uses an improved Asynchronous Advantage Actor-critic (ATSI-A3C) approach for multi-cloud infrastructure. The algorithm focuses on segmenting tasks and distributing sub-tasks to VMs based on their characteristics, aiming to minimize makespan, energy consumption, and resource costs.

Ghasemi et al. [22] present two novel hybrid approaches, VMPMFuzzyORL and MRRL, for optimizing virtual machine placement (VMP) in cloud data centers, addressing key challenges such as energy consumption, resource wastage, and load balancing. VMPMFuzzyORL integrates reinforcement learning (RL) with fuzzy logic to achieve precise placement but suffers from high execution time due to the fuzzy system. To overcome this, the MRRL approach combines k-means clustering of virtual machines with a multi-reward RL strategy, significantly reducing execution time while maintaining high efficiency in resource utilization and energy reduction. While these techniques are effective, it is also important to consider other factors in resource management, such as computational costs, data governance regulations, etc. Addressing these additional considerations is essential for developing a comprehensive and practical resource management solution.

As Table 1 shows, DynamoML and HYPREL excel at resource optimization, especially in task scheduling and GPU utilization within Kubernetes-based environments. However, they do not support multi-cloud infrastructure and lack the ability to enforce policies or manage compliance. Pangea offers multi-cloud support and good scalability, but it doesn't provide policy-driven control or compliance management, which are critical in enterprise environments. Sinan focuses on resource optimization for microservices with QoS targets but lacks support for multi-cloud, policy control, and compliance management. Our proposed framework excels in multi-cloud infrastructure with policy-based control, offering a more holistic solution with compliance and real-time monitoring.

Several researches have addressed the problem of multi-clouds. However, to our best knowledge, little attention has been devoted to general resource management in multi-cloud environment. It needs to manage many types of resources and select resource with flexibility on evaluation factor, criteria.

3.2 Application topology discovery

The field of application topology discovery is relatively new, particularly given the dynamic and distributed nature of deployments in the era of machine learning and DevOps. To achieve a better understanding of application deployment is a significant challenge. However, extensive research in network topology discovery and software architecture recovery has provided valuable insights. These research has been an inspiration in guiding our work on application topology discovery.

Gandalf [23] is a system designed for safe deployment in large-scale cloud infrastructures. It employs intelligent, end-to-end analytics to identify and mitigate potential issues

Table 1 Comparison of resource management frameworks

Framework/tool	Multi-cloud support	Resource optimization	Compliance management	Real-time monitoring	Scalability
DynamoML	No	Yes (GPU usage)	No	No	Moderate
HYPREL	No	Yes (Task placement)	No	No	Moderate
Pangea	Yes	Yes	No	No	High
Sinan	No	Yes (QoS)	No	No	Moderate
Proposed framework	Yes	Yes (Policy-driven)	Yes	Yes	High

before and during deployment. Gandalf uses anomaly detection, risk assessment, and automatic mitigation strategies to ensure deployment safety.

Khan et al. [24] emphasized that key steps in network topology discovery include identifying entities, controllers, and dependency services. This approach has inspired the discovery of application topologies. Hwang et al. [25] monitors network flows in data center and maps application topologies by grouping intensive connected VMs. They then classify the applications using a graph similarity algorithm. Similarly, Sangpetch et al. [26] utilize network traffic monitoring to analyze response times among components and reconstruct dependency relations. The previous works have suggested that dependency information can be revealed by network flow data, and it seems to be a reliable approach.

There is a considerable amount of literature on software architecture recovery, including a notable study by Thibaud Lutellier et al. [27]. This study addresses the impact of using more accurate symbol dependencies over traditional include dependencies to enhance the performance of architecture recovery techniques. The authors compared nine variants of six state-of-the-art recovery techniques on five open-source projects, including Chromium, ITK, Bash, Hadoop, and ArchStudio. Their findings indicate that symbol dependencies, which represent actual code relationships, lead to significantly higher accuracy in recovered architectures compared to include dependencies, which often inaccurately represent file-level dependencies. This work provides valuable insights into the dependency types and their effects on architecture recovery.

Zhang et al. [28] introduce SARIF (Software Architecture Recovery with Information Fusion) to enhance the accuracy of software architecture recovery. Unlike traditional methods that use limited information sources, SARIF integrates dependencies, code text, and folder structures, using techniques like TF-IDF, LDA, and community detection for a detailed analysis.

Link et al. [29] introduces RELAX, a novel method for recovering software architecture using text classification to address user-defined concerns. RELAX overcomes limitations of existing methods by ensuring scalability, efficiency, and deterministic results, which are critical for tracking architectural changes over time. It constructs architectural views from independent parts, facilitating large-scale system analysis, and provides an intuitive visualization of the architectural structure. Evaluations on open-source systems demonstrate that RELAX achieves higher accuracy and better scalability compared to other methods. The paper highlights RELAX's adaptability and effectiveness in maintaining the integrity of evolving software systems.

Link et al. [30] evaluates the utility and accuracy of three software architecture recovery (SAR) methods. The authors conclude that none of the methods fully meet the criteria for effective maintenance support, highlighting the need for improved SAR tools that combine the strengths of existing methods to provide reliable and meaningful architectural views for software maintenance.

In their study, Ullmann et al. [31] employ an architecture recovery approach to analyze and visualize subsystem coupling patterns in ten open-source game engines. By creating architectural models, the researchers identify key subsystems that frequently couple with each other. Their approach seems to be plausible to visualize architectural models of complicated software system by analyzing the relationships among subsystems.

Thibaud Lutellier et al. [32] evaluates the effectiveness of various software architecture recovery techniques using different types of code dependencies. The study compares nine variants of six recovery techniques on five open-source projects. The researchers found that using accurate symbol dependencies significantly improves the accuracy of recovered architectures. This paper demonstrates the critical role of dependency accuracy in the quality of architecture recovery. The main limitation of this software architecture recovery is that the recovered model does not reflect the real information of running application topology. However, code analysis seems to be a realistic method to obtain useful information about the dependency and connections of the application.

As Table 2 shows, SARIF and RELAX focus on recovering software architectures through static analysis, making them useful for offline evaluation but not for real-time monitoring. Pangea offers partial dynamic topology discovery but does not provide real-time monitoring or compliance enforcement across clouds. Our proposed framework supports dynamic, real-time topology discovery with compliance enforcement, making it ideal for monitoring and managing complex multi-cloud infrastructure.

In summary, while previous research offers valuable methods for network and software architecture discovery, there is a lack of comprehensive solutions for real-time application topology discovery in dynamic multi-cloud infrastructure. Our work addresses this gap by providing a topology discovery system tailored for monitoring

Table 2 Comparison of topology discovery systems

Framework/tool	Topology discovery type	Real-time monitoring	Compliance enforcement	Visualization
SARIF	Static	No	No	Yes
RELAX	Static	No	No	Yes
Pangea	Dynamic (partial)	No	No	Yes
Proposed framework	Dynamic (multi-cloud)	Yes	Yes	Yes

deployments in real-time, ensuring policy compliance and system integrity.

4 Proposed framework design

The proposed framework has three main components: a policy-based deployment management system, an agent-based deployment topology discovery system, and a graphic visualization engine to visualize the deployment topology. These three components address the critical phases of the ML application deployment process, from resource planning to monitoring. The policy-based deployment management system helps define and select resources for application deployment. The agent-based deployment topology discovery system can monitor the actual deployment, especially when an application is deployed independently by multiple teams. A hierarchical dashboard presents the topology at several levels. Along with existing industrial technologies, such as Gitlab CI[2] and Kubernetes, we believe that our proposed framework can manage the complete cycle of application deployment and operation.

4.1 Policy-based deployment management

The following subsections provide an overview of the proposed policy-based deployment management. To illustrate the overall deployment control, an illustrative process flow diagram is included. Subsequently, the key components of the policy-based deployment management, including the policy, policy engine, and request, etc. are introduced in detail.

4.1.1 Deployment management process

As shown in Fig. 1, the deployment management process has six key parts, deployer, deployment coordinator, policy engine, policy store, resource management gateway and resource domains. It is worth noting that the deployer could be a role in development team or a system component for automatic deployment.

The deployer sends deployment request to the deployment coordinator; The deployment coordinator sends the deploy conditions together with their user information to the policy engine. The policy engine is preloaded with policy files from policy store; The policy engine evaluates the deploy conditions and returns the resource domain; The deployment coordinator sends the resource requirement from the deployment request and the target resource

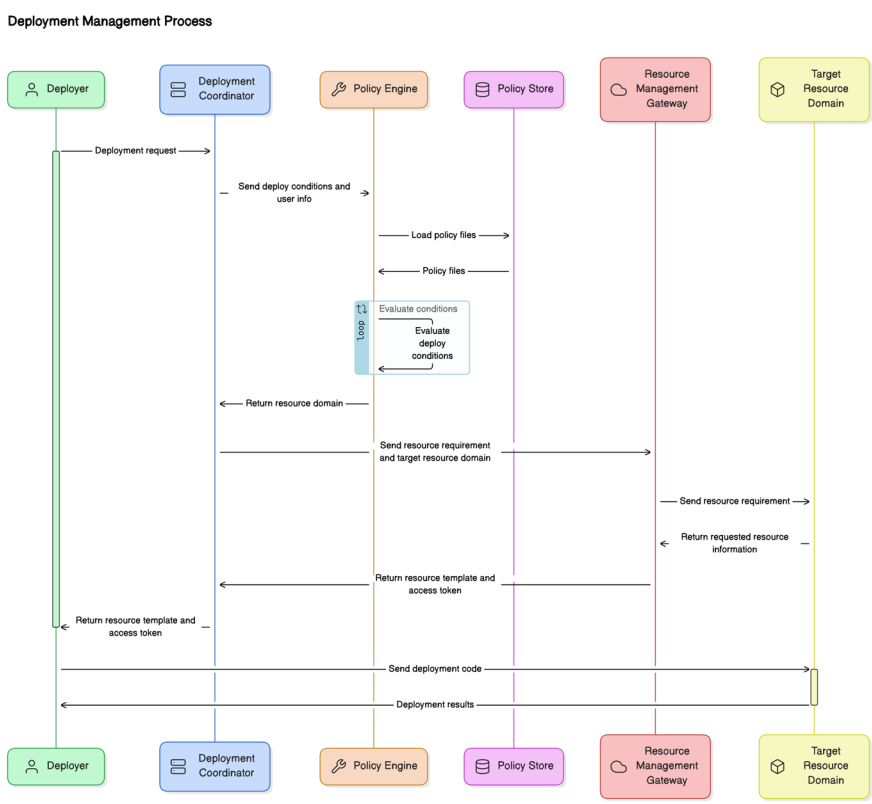


Fig. 1 Deployment management process

[2] https://docs.gitlab.com/ee/ci.

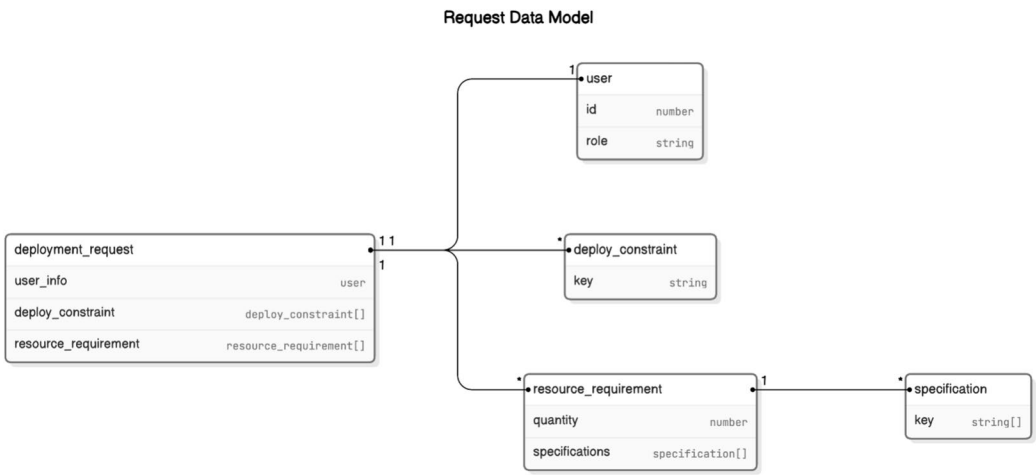
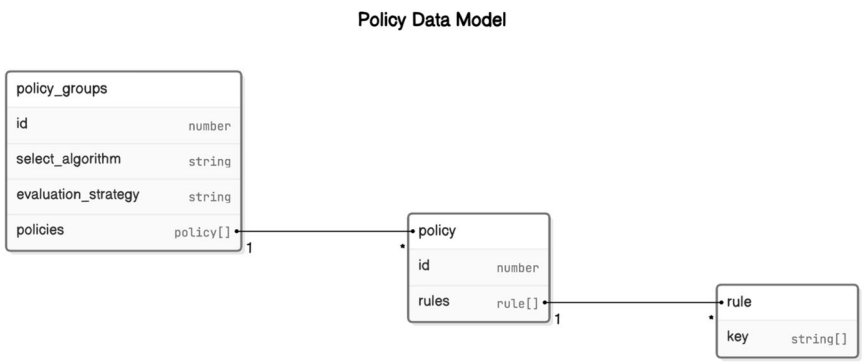


Fig. 2 Deployment request data model

Fig. 3 Policy data model



domain to the resource management gateway; The resource management gateway sends the resource requirement to the target resource domain; The target resource domain returns the requested resource information to the resource management gateway; The resource management gateway returns the resource template to the deployment coordinator; The deployment coordinator returns the resource template to the deployer; The deployer sends the deployment code on the target resource domain. The deployment then starts.

4.1.2 Request and policy data model

As shown in Fig. 2, the deployment request has three types of information, user information, deploy constraints, and resource requirement. User information describes the user who sends the request. It can be used for authentication and authorization of permitted resources. Deploy constraints describe the types and conditions of resources, such as location, computational cost, etc. Resource requirement defines the quantity and specifications of the resources. One deployment request can contain multiple sub-requests. The resource template has information on the requested resource and the corresponding configurations.

One policy contains several rules; each rule is a key-value pair. The key is the condition, and the value is the name of the corresponding target infrastructure domains. More than one policy can be grouped into policy groups. The policy data model is shown in Fig. 3.

In the policy group, the select algorithm and evaluation strategy can be defined. The select algorithm is useful when more than one domain is returned from the policy engine. The select algorithm selects one or more domains from the returned domains. Its options are random and distribute. The random option randomly selects one domain. The distribute option keeps the returned domains, later the resource requirement will be equally distributed to the domains. The evaluation strategy is useful when the request has multiple sub-requests. This algorithm has two options, strict and easy. The strict option only returns the success response when all subrequests are successful. The easy option returns success response from any successful subrequests.

4.1.3 Policy engine

The policy engine is a vital component in managing deployment requests and ensuring that they adhere to organizational policies and business constraints. The workflow begins

```
Data: Set of deploy constraints C = (c_1, c_2, ...c_n) n is the number of conditions
Result: Set of infrastructure domains D = (d_1, d_2, ...d_m) m is the number of domains
Initialization
PolicyHashMap p = loadPolicyHashMap()
B ← {} /* Initialize frequency map                                    */
for constraint c_i ∈ C do
    DA_i = evaluateRequest(p, c_i)
    /* DA_i is returned domain array for the condition c_i            */
    for domain d ∈ DA_i do
        if d ∈ B then
            B[d] ← B[d] + 1;
        else
            B[d] ← 1;
D ← {} /* Initialize result set                                       */
for domain d ∈ B do
    if B[d] = n then
        D = addToResult(d)
return D
```

Algorithm 1 Policy engine evaluation algorithm

when the policy engine receives a deployment request. The user information is used to authenticate the requester who makes the request and to verify their permissions.

Authentication and authorization are the first steps in the workflow. The engine checks the user's roles and permissions to ensure they have permission to request certain resources.

Following user verification and permission check, the engine evaluates each deploy constraint in the request against these predefined rules. This includes checking for compliance with specified constraints such as location, computational cost, and other conditions. If all constraints match the policy rules, the request is approved. However, if any constraint fails to comply, the request is denied.

Based on policy and request data model, we designed a policy evaluation algorithm as shown in Algorithm 1. To evaluate the request, the algorithm iterates each condition of the request. For each condition, the algorithm finds the eligible domains which defined in policy. The results from all conditions are counted. Then the algorithm iterates through the counted set and returns an array of intersected domains. If no intersected domains, return null.

Before request evaluation starts, policy engine needs to preprocess stored policy files. Policy engine first retrieves policy files. The engine then processes the policy files and create a hash map where keys are rule values (e.g., location=EU, resource-type=cpu) and values are sets of eligible domains (e.g., aws-eu-central-1).

Algorithm 1 operates by iterating over a set of deployment constraints C and evaluating each constraint against policy hash map p to determine suitable infrastructure domains. The evaluation step has a time complexity of $O(1)$, where for each constraint c_i, the policy engine checks it against p. After evaluating the constraints, the algorithm performs a counting operation, with a time complexity of $O(N_r)$, where N_r is number of all returned domains. The final step involves finding the intersection of domains, which also takes $O(k)$, where k is number of unique domains in B, where B is a frequency map to count domain occurrences. Thus, the overall time complexity of the algorithm is $O(N_r + k)$. Given these factors, the algorithm's performance is largely influenced by the number total returned domains from evaluation. The time complexity analysis suggests that for moderate values of N_r and k, the algorithm can perform efficiently.

Logging and auditing are integral parts of the workflow, ensuring transparency and accountability. The policy engine logs the details of the request, the evaluation process, and the final decision.

A small security module also enforces encryption for data at rest and in transit. The policy files are stored using AES-512 encryption, while all service communications–including agent uploads and dashboard queries are protected via TLS. This ensures security of deployment data.

Finally, the policy engine generates a response indicating whether the request is approved or denied. If the request is approved, the response includes available resource domains.

4.1.4 Resource management gateway

Resource management gateway is responsible to contact resource domain and get corresponding resource information from each resource domain. It has plugin based architecture to adapt to various resource types, such as VM cluster, container cluster. For example, for resource domain represents a Kubernetes cluster a customized plugin can be implemented to send a request to Kubernetes cluster.

In addition to VM and container resource, the resource management gateway provides limited support for serverless architectures. It integrates serverless services (e.g., AWS Lambda, Azure Functions) as resource domains, allowing allocation of serverless compute resources based on policies. However, due to the lack of transparency in serverless environments, the topology discovery engine (Sect. 4.2) currently cannot support serverless architectures.

4.2 Deployment topology discovery

As another important component of our research, deployment topology discovery is designed to automatically show the topology of ML application deployment. The topology discovery plays a crucial role in enforcing the policy-based deployment control within this framework. By continuously monitoring the deployment topology, it ensures that all activities comply with predefined policies, providing a mechanism to detect and prevent any non-compliant actions. This guarantees that deployments adhere to organizational or regulatory requirements, safeguarding against policy violations.

In addition to ensuring compliance, the topology discovery system is essential for managing the complexity of multi-cloud deployments. It tracks dependencies between application components, ensures the integrity of distributed systems, and provides operational transparency. Furthermore, it assists in debugging by identifying the source of issues in real time, and offers clear visibility into the relationships between various resources across multiple clouds, making it a critical tool for efficient multi-cloud management.

We aim to efficiently provide application runtime information as simple as possible. At meantime, we want to minimize the additional workload introduced by this component while preserving the autonomy of each application team.

This component has three key parts, data collection agent, topology discovery engine and graphic visualization engine. The data collection process is shown in Fig. 4. The data collection agent collects and pushes the application and the application network information. The data collection agent preprocesses the information collected. The data collection agent sends data to topology discovery engine. Data from multiple agents are merged and re-indexed in topology discovery engine then saved in database. The graphic visualization engine visualizes data from database.

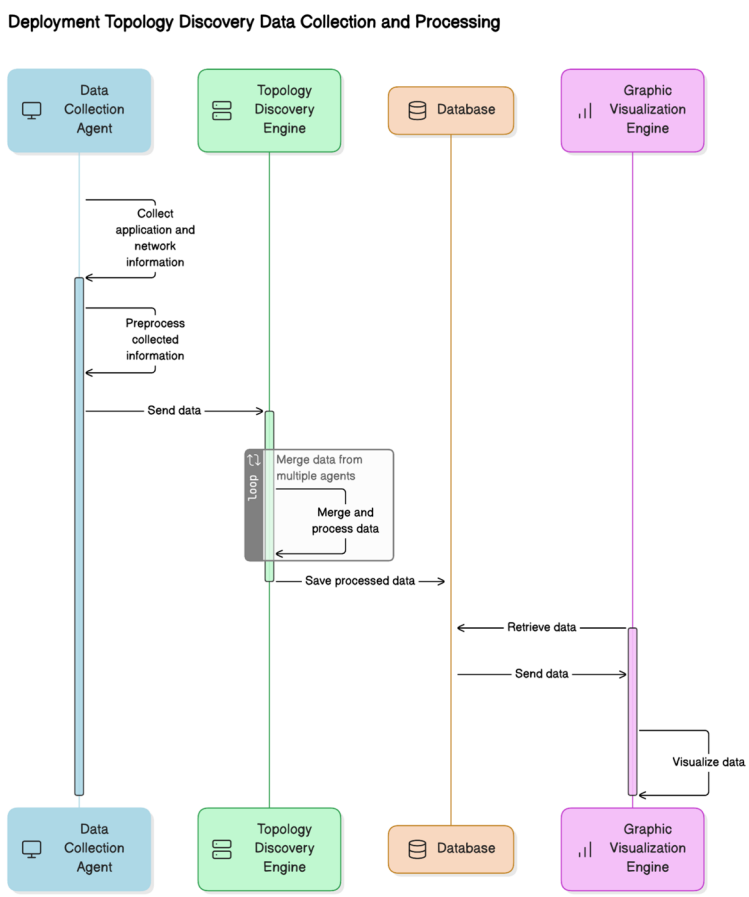


Fig. 4 Deployment topology discovery data collection and processing

4.2.1 Data collection agent

Data collection agent has hybrid mechanism combining event-driven and pull and push approaches to collect application data and sends to topology discovery engine. It is It is activated both by state changes (e.g., pod lifecycle events) as well as by proactively pulling data from running application within defined time interval, preprocessing data, attaches with timestamp then compresses it. Finally, the agent pushes data to the preconfigured topology discovery engine address. The workflow of the agent is shown in Fig. 5. To ensure consistent updates, if no state changes are detected within a given interval, the agent defaults to executing at least one pull-and-push cycle during that period.

The data collection agent first collects application name related information. This information will later be used to identify different applications in the same resource domain. In order to do so, there is a requirement for all applications naming conventions. Each application name or application service name must end with an application ID. This application ID will be used to aggregate the distributed application deployment information in topology discovery engine.

at the same time, network traffic data is collected. It is only necessary to collect listening ports, established connections and process data about active processes on the host. This information is crucial for constructing the topology graph. Host related metadata is also collected by agent.

All the collected data is then sent to a preprocessing stage. During this phase, timestamps are added to the data and compressed for efficient storage and transmission.

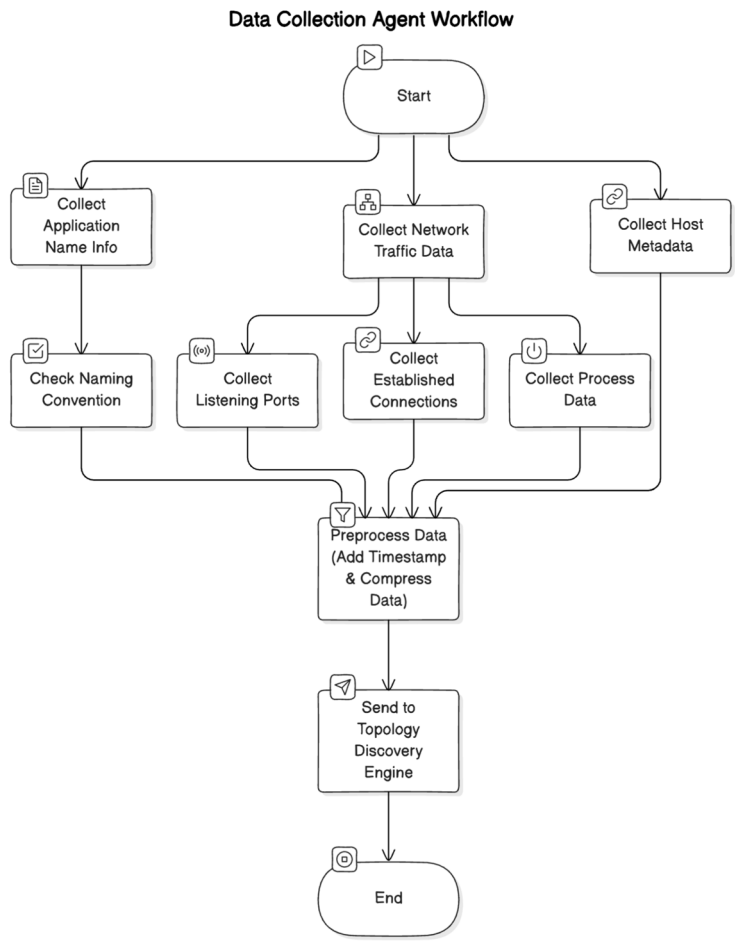
Finally, the preprocessed data is sent to the topology discovery engine, which is responsible for analyzing the data to understand the topology and relationships among different entities.

4.2.2 Topology discovery engine

The topology discovery engine receives data from multiple data collection agents throughout the infrastructure. Its primary responsibility is to process these data and generate topology graphs. Its workflow is shown in Fig. 6.

The topology discovery engine has an important mechanism known as the data aggregation buffer, which is responsible for handling and organizing the incoming data from multiple collection agents. Within a specified buffer time window, all received data timestamps are checked and reformatted. This step is important to ensure that the data aligns temporally. Any data with timestamps falling

Fig. 5 Data collection agent workflow diagram



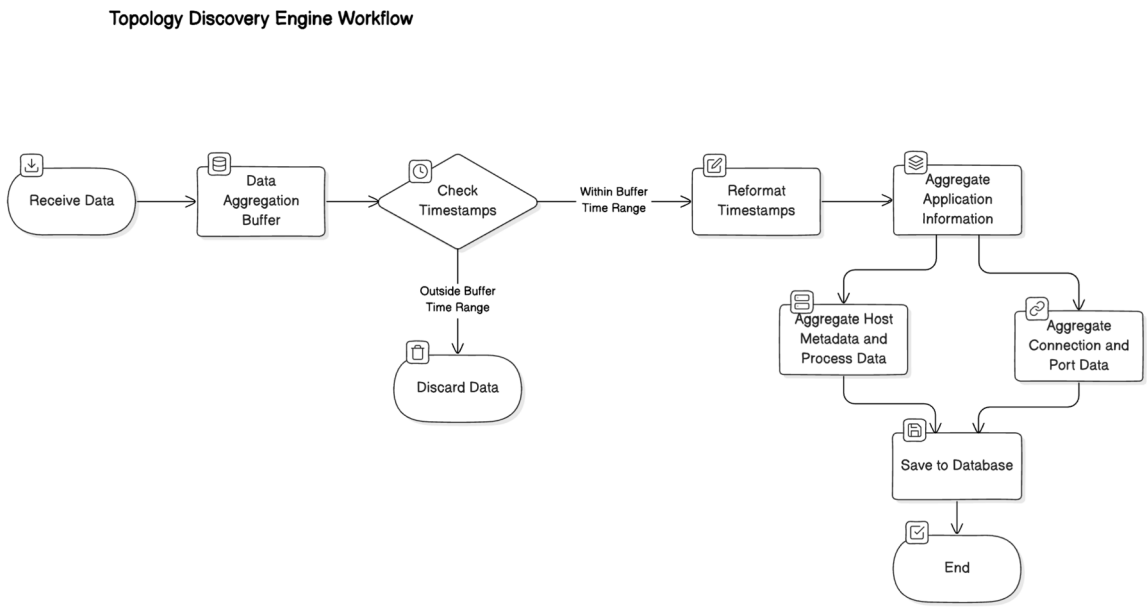


Fig. 6 Topology discovery engine workflow diagram

outside this predefined buffer time range is automatically discarded to maintain the consistency and accuracy of the topology graphs. This timestamp verification process prevents outdated data from providing incorrect deployment information.

In the next step, the engine first aggregates application information. This phase is critical as it identifies the data based on the application it belongs to. Following this, host metadata and process data are combined and aggregated to form distinct entities within the topology graph. These entities represent individual components of the application.

Next, the data related to established connections and listening ports is merged and aggregated. This phase focuses on forming the connections among the previously defined entities.

Finally, the generated data, comprising both entities and their connections, is saved in a database. The graphic visualization engine will later retrieve data from the database to render the topology graphs.

4.2.3 Graphic visualization engine

Graphic visualization engine queries data from database and renders the graphs. These graphs are refreshed in live time. This ensures that the visual representation reflects the most current status of a running application.

The engine offers two layers of view. The first layer displays a connected graph which presents the deployment nodes (machines or containers) connected in application and the connections among them. The second layer is the view of single node. It shows the processes running on one node and the ports which are ready to receive connection.

5 Experiments and case study

5.1 Prototype architecture

This section provides a detailed description of the prototype architecture for the proposed policy-based deployment management and topology discovery in a multi-cloud MLOps environment. The architecture integrates various self-developed modules and existing open-source software systems to manage ML application deployment and operation efficiently. In order to demonstrate our proposed framework, we built multi-cloud environment on Azure and AWS. We host our framework in AWS. An innovative multi-clouds Kubernetes cluster management system, Karmada[3], is employed to facilitate our experiments cross clouds. In one of Kubernetes clusters in AWS, we deployed a modified version of KubeFlow[4] as machine learning workflow automation platform. We modified the code to let our framework directly manage the Training Operator and KServe deployments.

As shown in Fig. 7, the architecture consists of two primary components: policy-based deployment management service and deployment topology discovery service.

These components are built using both self-developed modules (solid lines in diagram) and existing open-source tools or cloud services (dotted lines). The focus of this section is on the custom-developed modules depicted with solid lines in the architecture diagram.

[3] https://karmada.io.

[4] https://www.kubeflow.org.

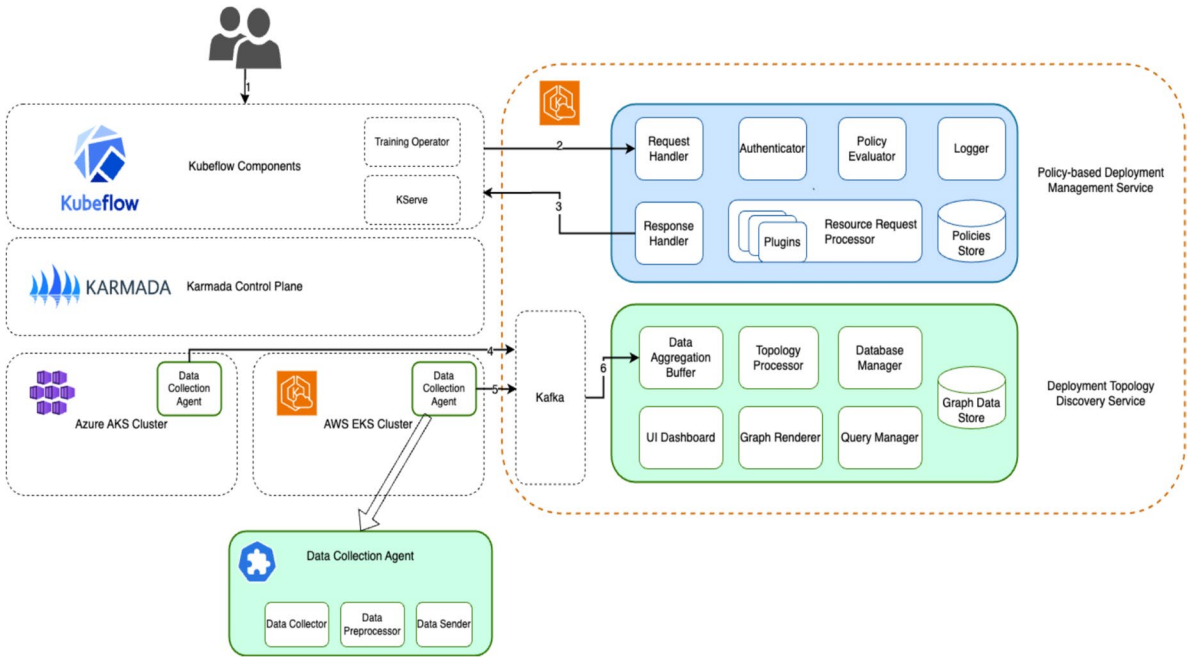


Fig. 7 Framework prototype architecture

5.1.1 Policy-based deployment management service

As illustrated by the blue component in Fig. 7, the policy-based deployment management service is responsible for orchestrating deployment requests according to predefined policies.

Request handler manages the incoming deployment requests from deployers or automated systems. It is built using Python with Flask to handle HTTP requests. It validates request data and forwards it to the policy engine.

Authenticator verifies the identity and permissions of the requestor. It utilizes JWT for token-based authentication and role-based access control (RBAC) for authorization.

Response handler processes the results from the policy engine and resource management gateway, formatting the response for the deployer. It is implemented in Python to ensure seamless integration with the request handler and policy engine.

Policy evaluator evaluates deployment requests against predefined policies to determine the appropriate resource domains. It uses a custom policy evaluation algorithm written in Python. Policies are stored in YAML format and loaded as needed.

Logger records all requests, evaluations, and responses for auditing and debugging purposes. It is integrated with a logging framework to capture detailed logs and store them in AWS CloudWatch logging service.

Resource request processor sends resource requests to various resource domains and aggregates the responses. It is designed with a plugin-based architecture in Python. Plugins are developed for different resource types, such as Kubernetes clusters and VM clusters. In this prototype, we use Karmada to manage multi-cloud Kuberntes clusters; therefore, the plugin is developed for Karmada templates.

5.1.2 Deployment topology discovery service

Data collection agent, as shown by the green components in Fig. 7, collects application and network data from different clusters. Agents are written in Go for efficient data collection and transmission and deployed by Kubernetes Operator in this prototype. They include sub-modules for data collection, preprocessing, and sending.

The data collection agent retrieves application metadata via the Kubernetes API, monitors network traffic through Istio's sidecar that tracks active connections, and collects host-related information. Figure 8 shows an example of network data collected from agents.

Data preprocessor adds timestamps and compresses the data. Data sender sends the preprocessed data to the topology discovery engine via Kafka[5]. Kafka is used as a cache to buffer incoming data from possible large amount of data collection agents.

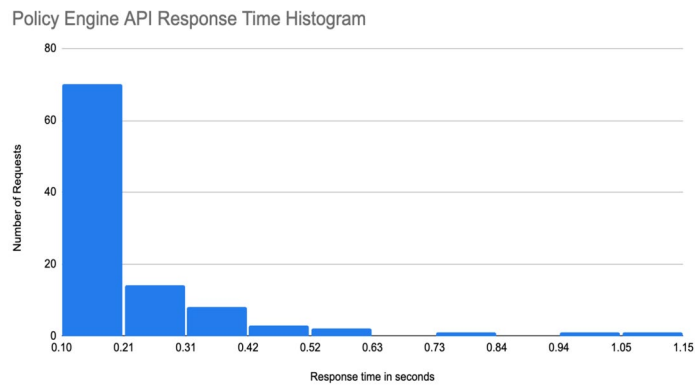
Communication between data collection agents and the topology discovery engine is secured using mutual TLS (mTLS), ensuring that both ends are authenticated and traffic is encrypted. In addition, Kubernetes Network Policies are applied to control ingress and egress at the pod level.

[5] https://kafka.apache.org.

Application Name	Node	Local IP	Port	Foreign IP	Port	Status
demoapp_rubrx	plant-train-4052-ps-0	10.244.0.4	2222	0.0.0.0	*	LISTEN
demoapp_rubrx	kserve-models--web-app-asia-5	10.96.2.15	5000	0.0.0.0	*	LISTEN
demoapp_rubrx	kserve-models--web-app-asia-5	10.96.2.15	15020	0.0.0.0	*	LISTEN
demoapp_rubrx	kserve-models--web-app-asia-5	10.96.2.15	15021	10.96.2.19	15012	ESTABLISHED
demoapp_rubrx	kserve-models--web-app-asia-5	10.96.2.15	15090	0.0.0.0	*	LISTEN
demoapp_rubrx	kserve-models--web-app-asia-5	10.96.2.15	15006	0.0.0.0	*	LISTEN
demoapp_rubrx	kserve-models--web-app-asia-5	10.96.2.15	15001	0.0.0.0	*	LISTEN

Fig. 8 Example of data collected from agents

Fig. 9 Policy engine response time histogram



Data aggregation buffer aggregates data from multiple data collection agents and ensures temporal consistency. This module uses Kafka Streams to buffer and align data temporally before processing.

Topology processor processes aggregated data to generate deployment topology graphs. It is written in Python, it constructs graphs based on application dependencies, network connections, and host metadata.

Database manager manages the storage of processed topology data. It uses MongoDB[6] to store graph data, ensuring efficient querying and retrieval.

Graph renderer renders visual representations of the deployment topology. UI dashboard provides a user interface to display topology graphs and allow user interaction. It is built with D3.js, periodically queries the database and updates the visualization.

Query manager handles user queries and retrieves the necessary data from the database. It is developed in Python with a RESTful API to facilitate data access.

5.2 Performance evaluation of policy engine

In this subsection, we describe an experiment designed to assess the QoS metrics of the component under realistic conditions. The primary objective was to measure the engine's response time when processing YAML requests against a set of pre-defined policies. Specifically, the experiment focused on the system's response time distribution under moderate load conditions, simulating real-world usage scenarios.

The results showed that the average response time per request was approximately within 0.10 to 0.20 seconds. This was expected based on the algorithm's time complexity and network setup, where the bulk of the processing time is spent evaluating constraints against policies. Sorting and intersection of domains, while necessary, had a minimal impact on overall performance.

5.2.1 Experimental setup

The policy engine was deployed as a single instance in the eu-central-1 region of AWS, close to the testing client region. A total of 1000 policies were generated, with random values for constraints such as CPU, memory, region, etc. Similarly, 100 requests were created with various constraints.

A shell script was used to automatically send these 100 requests to the policy engine for evaluation. The response time for each request was recorded in seconds and the results were aggregated to produce a histogram that illustrates the distribution of response times, as shown in Fig. 9.

5.2.2 Results and analysis

The histogram reveals that the majority of requests (approximately 70%) were processed within 0.10 to 0.20 seconds, indicating that the policy engine performs efficiently under the given load. However, there is a noticeable tail in the distribution, with a small number of requests taking up to 1.10 seconds to process. This variability may be due to the inherent nature of cloud environments, where occasional

[6] https://www.mongodb.com.

resource contention can cause longer processing times for some requests.

Despite the variability, the results suggest that the system performs adequately for moderate workloads, with a typical response time of less than 0.20 seconds for the majority of requests. This response time is suitable for the applications, where timely policy evaluation is critical.

While the experiment provides valuable insights into the performance of the policy engine, there are limitations to consider. The policy engine was deployed as a single instance without any auto-scaling or load-balancing mechanisms. In a real production environment, deploying multiple instances of the policy engine with an auto-scaling solution and load balancer would likely have better performance. An auto-scaling setup would allow the system to dynamically allocate resources based on incoming request load, potentially reducing the tail latency observed in this experiment.

5.3 Deployment management evaluation

In this section, we evaluate the effectiveness of the policy-based deployment management framework through a series of experiments conducted in a multi-cloud environment. The deployment of a machine learning application was tested across various infrastructure configurations, demonstrating the effectiveness and flexibility of the framework.

```
1   deployment_request:
2     user_information:
3       user_id: "87654321"
4       user_role: "application_owner"
5     deploy_constraints:
6       - location: "east-us"
7         computational_cost: "high"
8         resource_type: "gpu"
9     resource_requirements:
10      - quantity: 3
11        specifications:
12          gpu: 2
13          memory: 32GB
14          cpu: 8
15          resource_type: "Kubernetes"
16          pod_specifications:
17            containers:
18              - name: "training-container"
19                image: "training-image"
20                resources:
21                  limits:
22                    cpu: "8"
23                    memory: "32Gi"
24                    nvidia.com/gpu: "2"
```

Fig. 10 Model training request file

5.3.1 Deployment environment and infrastructure

To evaluate the framework, we deployed three Kubernetes clusters across different cloud platforms: Azure and AWS. The infrastructure consisted of:

- Azure Kubernetes Cluster (east-us, East US): Utilized ND-series VMs with Nvidia Tesla P40 GPUs for computationally intensive model training tasks.
- AWS Kubernetes Cluster (eu-central-1, Frankfurt): General-purpose VMs used for model serving in proximity to European end-users.
- AWS Kubernetes Cluster (ap-east-1, Hong Kong): General-purpose VMs used for model serving near Asian end-users.

These clusters allowed us to evaluate how the framework manages deployment across different regions, using both GPU and CPU resources for training and serving purposes.

The deployment uses several resilience features aligned with production best practices. First, Kubernetes-native liveness and readiness probes are configured for core components such as the policy engine and topology discovery engine. These probes monitor service health and automatically trigger restarts if a component becomes unresponsive or fails initialization checks. Second, automated daily snapshot backups are implemented. These snapshots, managed using cloud-native backup tools, support point-in-time recovery. Third, the framework is deployed across multiple availability zones within each cloud provider. This zonal distribution ensures continued operation if a particular zone experiences service disruption. In the event of a failure, the policy engine can still process deployment requests from unaffected zones, and topology discovery remains functional with partial data input. Together, these mechanisms provide a strong resilience strategy that improves system reliability in multi-cloud environments.

5.3.2 Application deployment

This evaluation deploys a machine learning application based on the PlantVillage dataset. The application consisted of two key phases.

Model Training: A VGGNet model [33] was trained using the PlantVillage[7] dataset. This phase was GPU-intensive and executed on the Azure Kubernetes cluster, leveraging its ND-series GPU resources. The request file (Fig. 10) specified the necessary resources and configurations for this task.

[7] https://www.kaggle.com/datasets/emmarex/plantdisease.

Model Serving: Once the model was trained, it was deployed to AWS Kubernetes clusters for serving predictions. The model was distributed across the Frankfurt and Hong Kong clusters to reduce latency and improve availability for European and Asian users, respectively. The request file for model serving (Fig. 11) detailed the deployment requirements, ensuring geographic proximity to the end-users.

Figure 12 illustrates the application workflow that integrates a machine learning pipeline within a CI/CD pipeline. The workflow begins with source code management in Gitlab, triggering the pipeline, which includes a CI step before initiating the ML pipeline. CI step validates code, sends request to policy engine to obtain resource domain and builds infrastructure code. Then the ML pipeline runs in the resource domain and it consists of data preprocessing, model training, evaluation, and validation, where models that meet predefined performance thresholds are registered to the model registry. Once an ML model is registered, the deployment process is controlled by manual approval, allowing for feedback from the team. The deployment stage interacts with the policy engine again to obtain deployment resource domain. Then model is served via an inference endpoint. In our experiment, CI/CD pipeline and data are hosted in AWS, while machine learning pipeline is deployed in Azure.

While our evaluation executed on a plant disease classification workflow, the framework itself is domain-agnostic. Ongoing work involves applying it to other scenarios, including multi-region financial forecasting and healthcare model deployment. These efforts aim to validate its adaptability across diverse contexts.

```
deployment_request:
  user_information:
    user_id: "87654321"
    user_role: "application_owner"
  deploy_constraints:
    - location: "eu-central-1"
      computational_cost: "medium"
      resource_type: "cpu"
    - location: "ap-east-1"
      computational_cost: "medium"
      resource_type: "cpu"
  resource_requirements:
    - quantity: 12
      specifications:
        cpu: 1
        memory: 2GB
        resource_type: "Kubernetes"
        pod_specifications:
          containers:
            - name: "serving-container"
              image: "serving-image"
              resources:
                limits:
                  cpu: "1"
                  memory: "2Gi"
```

Fig. 11 Model serving request file

5.3.3 Policy file and resource allocation

Both the training and serving phases were deployed following a predefined policy file (Fig. 13), which specified computational and geographical resource requirements. The policy file specified the resource requirements for computation (GPU for training, CPU for serving) and the geographical locations (East US for training, Europe, and Asia for serving).

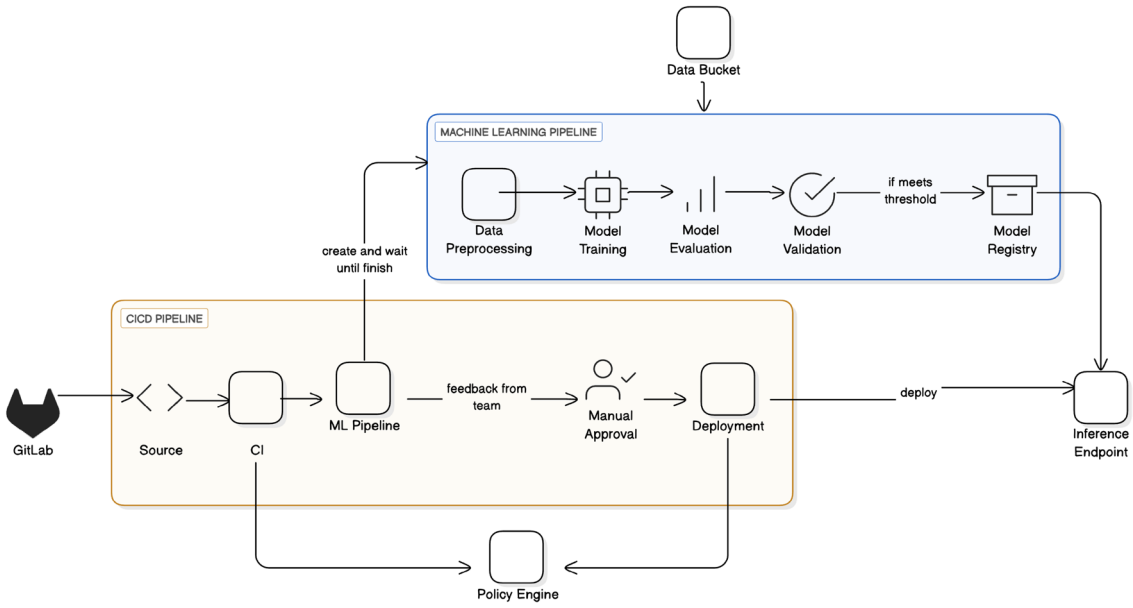


Fig. 12 Application workflow integrating CI/CD pipeline

```yaml
policy_groups:
  - name: example_policy_group
    id: 8372
    select_algorithm: distribute
    evaluation_strategy: strict
    policies:
      - name: example_policy_us
        id: 194835
        rules:
          location:
            - "us"
            - "east-us"
          computational_cost: "high"
          resource_type: "gpu"
        target_infrastructure_domains:
          - azure_east_us
      - name: example_policy_eu
        id: 460129
        rules:
          location:
            - "eu"
            - "eu-central-1"
          computational_cost: "medium"
          resource_type: "cpu"
        target_infrastructure_domains:
          - aws_eu_central_1
      - name: example_policy_asia
        id: 732614
        rules:
          location:
            - "asia"
            - "ap-east-1"
          computational_cost: "medium"
          resource_type: "cpu"
        target_infrastructure_domains:
          - aws_ap_east_1
```

Fig. 13 Policy file

```yaml
deployment_response:
  status: success
  sub_requests:
    - request_id: "req-training-1"
      status: success
      selected_domain: "azure_east_us"
      resource_allocation:
        - resource_id: "resource-001"
          location: "east-us"
          gpu: 2
          cpu: 8
          memory: 32GB
          labels:
            target_resource_domain_name: "azure_east_us"
      credentials:
        username: "azure_user"
        password: "azure_password"
        token: "azure_token"
```

Fig. 14 Model training response file

Upon execution of the deployment requests in MLOps build pipeline, we received confirmation of successful resource allocation for both training and serving phases sequentially. As shown in the first response in Fig. 14, the training request (req-training-1) was successfully processed. The Azure cluster in east US was selected as the domain for resource allocation. Credentials for accessing this resource domain were also provided, ensuring secure and authenticated access.

```yaml
deployment_response:
  status: success
  sub_requests:
    - request_id: "req-serving-1"
      status: success
      selected_domain: "aws_eu_central_1"
      resource_allocation:
        - resource_id: "resource-002"
          location: "eu-central-1"
          cpu: 1
          memory: 2GB
          labels:
            target_resource_domain_name: "aws_eu_central_1"
      credentials:
        access_key: "aws_access_key_eu_central"
        secret_key: "aws_secret_key_eu_central"
        session_token: "aws_session_token_eu_central"
    - request_id: "req-serving-2"
      status: success
      selected_domain: "aws_ap_east_1"
      resource_allocation:
        - resource_id: "resource-003"
          location: "ap-east-1"
          cpu: 1
          memory: 2GB
          labels:
            target_resource_domain_name: "aws_ap_east_1"
      credentials:
        access_key: "aws_access_key_ap_east"
        secret_key: "aws_secret_key_ap_east"
        session_token: "aws_session_token_ap_east"
```

Fig. 15 Model serving response file

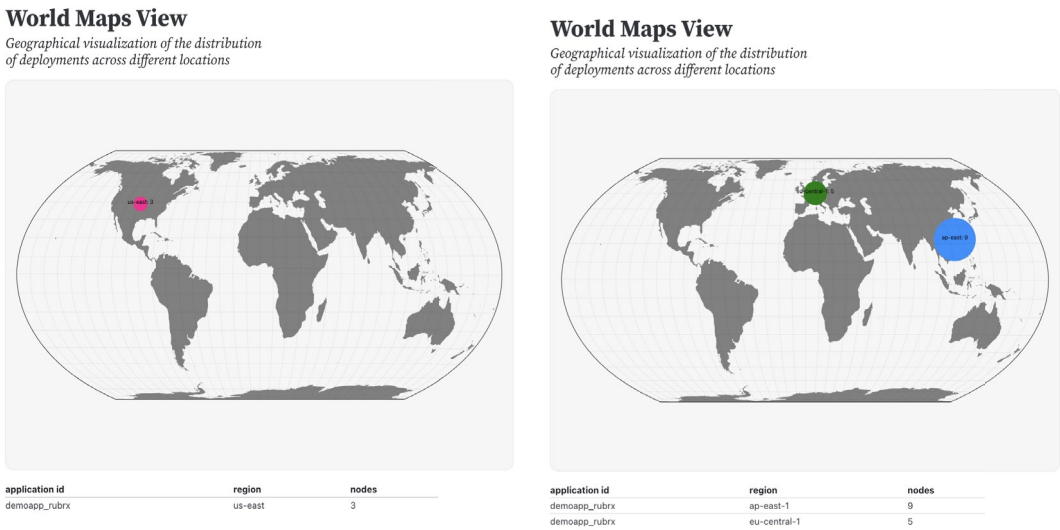
For the serving requests, two separate responses were received as shown in Fig. 15. These responses confirm that the specified resources were successfully allocated in accordance with the defined policies.

A critical component of these responses is the 'labels' field. These labels are crucial as they indicate the target resource domain name, which Karmada, our multi-cloud orchestration tool, uses to propagate the pods to the specified clusters.

5.3.4 Analysis of Results

The evaluation results indicated successful execution of both the model training and serving phases. The response files (Figs. 14 and 15) showed the correct assignment of resources, with the policy engine selecting the appropriate clusters based on the defined criteria.

However, the topology discovery engine highlighted an issue: the Hong Kong cluster deployed two more nodes than initially requested. This discrepancy was identified in the world map chart (Fig. 16b), where 14 nodes were deployed instead of the 12 specified in the request file (Fig. 10). Given that a few engineers on the project have access permissions to the cluster, the number of pods may occasionally be altered under certain circumstances. This example demonstrates the value of the topology discovery feature, which provides a real-time visual representation of deployment distributions and can detect deviations from the planned configuration.



(a) World map training (b) World map serving

Fig. 16 World map chart

Overall, the policy-based deployment framework successfully managed the resource allocation and deployment of the machine learning application across a multi-cloud infrastructure, optimizing deployment automation while maintaining compliance with specified policies.

This strategic allocation ensures improved utilization of resources and performance of the machine learning application across different geographical regions. The policy file facilitates automated decision making for resource allocation, ensuring that the right type of resources are used for the appropriate tasks.

5.4 Topology discovery evaluation

We have built three types of dashboard charts that collectively provide comprehensive monitoring and visualization, enhancing our ability to manage and optimize the deployment of our machine learning applications.

5.4.1 Map chart

The topology dashboard with a world map chart offers a geographical visualization of the distribution of deployments across different locations, providing a clear visual representation of the global spread of our deployments. This map-based overview facilitates easy identification of deployment locations. As illustrated in Fig. 16, the distributed TensorFlow training has three pods running in the North America region. In contrast, application web app deployments are located in the EU and East Asia, with five and nine pods, respectively.

As we noticed in Fig. 16b, the total nodes are 14 (9 in ap-east-1 and 5 in eu-central-1), while in request file in Fig. 11 only 12 nodes are requested. Therefore, we identified an issue that with granted credential the team has deployed 2 more nodes than planned.

5.4.2 Network chart

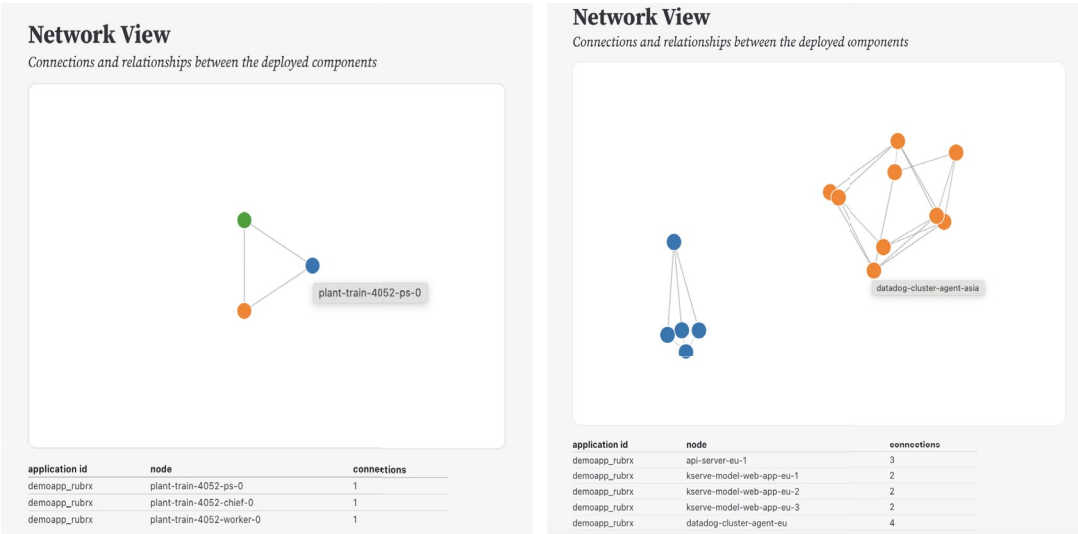
The network chart in the topology dashboard illustrates the connections and relationships between the deployed components, showing the interactions among various elements. This visualization helps understand the architecture of the deployed system, including data flow and interactions between different components or services. Additionally, it aids in identifying potential bottlenecks or points of failure.

In Fig. 17a, the network chart shows the distributed TensorFlow training setup, highlighting the parameter server (PS), the chief, and the worker nodes. Figure 17b depicts two application deployments. In the first deployment (shown in blue), one API server connects to three application backend. In the second deployment (shown in orange), one database connects to all application backend nodes, highlighting a more complex interaction pattern.

5.4.3 Node view chart

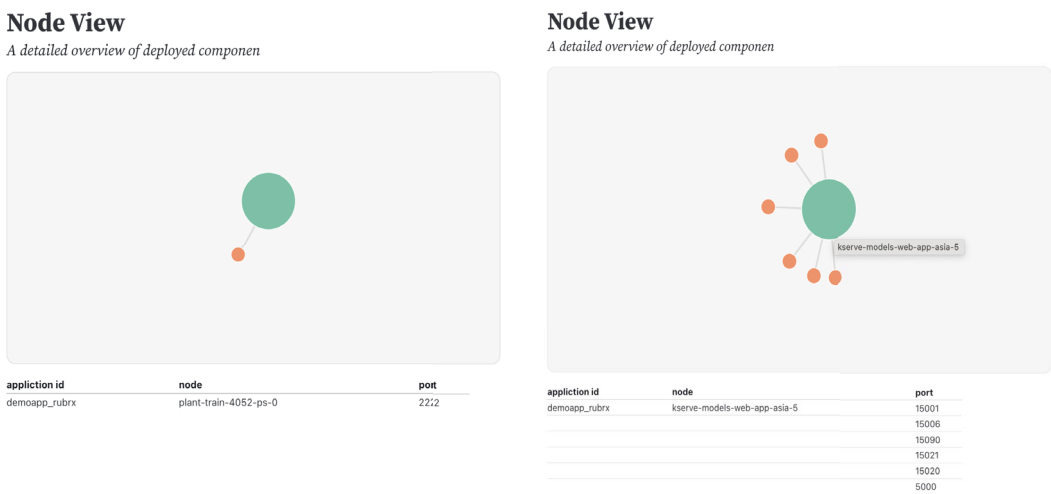
The node view dashboard provides a detailed overview of each deployed component, which is essential to debugging and examining the details of each deployment component. This detailed node-level view allows for the quick identification of failed services or components.

In Fig. 18a, the node view chart shows a single training pod"plant-train-4052-ps-0", where we can see the TensorFlow gRPC port 2222. Figure 18b shows an application pod"kserve-model-web-app-asia-5", where, in addition to



(a) Network training (b) Network serving

Fig. 17 Network chart



(a) Node view training (b) Node view serving

Fig. 18 Node view chart

port 5000, we can see Istio[8] related ports such as 15020, illustrating the various ports utilized by the application for different functionalities.

5.4.4 Conclusion

Collectively, these visualizations provide a detailed assessment of the deployment's topology across different regions, nodes, and network connections. By using these charts, we were able to identify deployment inconsistencies and analyze the interconnections within the multi-cloud infrastructure, providing valuable insights for optimizing resource allocation, detecting potential issues. Experiments also show the hybrid mechanism of data collection agent's design emphasizes a small footprint and efficient algorithms to minimize its impact on the host system. By optimizing data collection and transmission, the topology discovery engine provides accurate and timely insights into application operation without compromising performance.

5.5 User feedback collection

In the proof-of-concept development phase, user feedback is collected and managed through GitLab's integrated issue feature. This feature is open for internal users as well as project development teams. Users, including data scientists and ML engineers, can submit feedback directly via GitLab issues, which are categorized into predefined labels such

[8] https://istio.io.

as "policy compliance", "resource allocation", and "topology visualization". This feedback mechanism makes sure that feedback and improvement ideas are directly informed by users. Future work will integrate AI-driven analytics to automate the extraction of actionable insights from user feedback and log files.

A feedback feature was also added into the dashboard, allowing users to rate deployment success (1–5 stars) and submit free-text feedback. It then categorizes feedback into predefined tags (e.g., 'latency', 'policy violation') for future improvement.

6 Discussion

In this paper, we have presented a policy-based framework for multi-cloud infrastructure and demonstrated its capability with several deployment scenarios. The results from our experiments indicate the framework's effectiveness in managing resources and ensuring compliance with defined policies.

As shown, our results demonstrate that application deployment is regulated by policies grounded in both business and technical considerations. These policies control the location of applications and their data within heterogeneous cloud infrastructure, ensuring that deployments adhere to specific requirements.

The prototype we developed has visualized the real distribution of applications, highlighting unexpected changes. This capability is particularly valuable for understanding distributed applications frequently deployed across multiple clouds. The framework provides an abstract layer that simplifies the management of complex deployment requirements and cloud infrastructure.

By introducing automation, it reduces the workload of deployers, allowing them to focus on developing deployment scripts rather than understanding business regulations and identifying the appropriate resource domains.

A key advantage of this framework is the separation of responsibilities between deployers, infrastructure administrators, and business decision-makers. Deployers can declaratively specify application deployment requirements, while the policy engine determines the correct infrastructure domain, and the infrastructure broker locates the necessary resources. The topology discovery component can generate an overall application distribution graph, offering insights into running applications. This is particularly beneficial for large applications composed of multiple services, where deployment scales can range from dozens to thousands of nodes.

However, we recognize two limitations in our research. The first limitation is the restricted support for serverless architectures. Serverless services are increasingly popular due to their automatic scaling capabilities and pay-as-you-go pricing model. However, our proposed framework currently provides only limited support for it. While the plugin-based resource management gateway can provide policy-driven allocation of serverless services, the topology discovery engine cannot collect enough data from these services. This is because serverless architectures abstract underlying infrastructure details, limiting the framework's ability to accurately reconstruct application topologies.

The second limitation is the computational overhead introduced by the data collection agent. The agent-based topology discovery periodically polls application metadata, network traffic, and host information at fixed intervals. In large-scale or resource-constrained environments, this process can impose a significant CPU and memory overhead, particularly on nodes running compute-intensive workloads. Additionally, transmitting telemetry data to the central topology discovery engine consumes network bandwidth, potentially affecting performance. A preliminary solution for this involves allocating additional computing resources to the data collection agents and deploying the topology discovery engine closer to the application to reduce latency. This solution, however, might hinder the scalability of the framework.

Despite these limitations, the experiments lead us to conclude that the proposed approach represents a step toward more structured and policy-driven ML operations in multi-cloud environment.

7 Conclusion and future work

The proposed framework for MLOps in a multi-cloud environment demonstrates potential for improving the efficiency and effectiveness of ML model deployment and management. By addressing key challenges in resource provisioning and topology discovery, the framework offers a structured approach that can be adapted to various industries and use cases. However, ongoing research and development are necessary to address its limitations and enhance its capabilities, ensuring that it can meet the evolving requirements of modern enterprises.

Future research could explore several aspects to build on the current work. First, we recognize that broader validation through diverse case studies would further improve the framework. As part of our future work, we plan to collaborate with domain experts to conduct additional experiments across various application types. Second, integrating AI-driven analytics to automate the extraction of actionable insights from user feedback and log files could enable more data-driven decision-making. Third, expanding the

framework to support a broader range of cloud infrastructures and emerging technologies, such as serverless architectures, could broaden its applicability. As organizations continue to innovate and adopt new technologies, the framework will need to remain flexible and scalable.

Acknowledgements The authors would like to thank Dr. Jingzhou Sun's kind support and valuable help during this research.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature. The authors declare that no funds were received during the preparation of this manuscript.

Data availability Due to the ongoing status of this research and institutional policies, the project source code is not publicly available at this time. This project is currently only implemented as a proof of concept and is not yet production-ready. Additionally, project documentation is still under development, which may limit usability at this stage. However, to support validation and reproducibility, we have taken the following steps. Detailed methodology description, the paper provides comprehensive details on the framework's architecture, algorithms, and some implementation details to facilitate replication. Selective code sharing, researchers who wish to access specific portions of the implementation for academic and non-commercial research purposes may send a request to the corresponding author. Future open access commitment, we plan to open source relevant portions of the source code upon completion of research phases and after addressing documentation gaps and security considerations. We believe these measures provide sufficient transparency and reproducibility. Researchers interested in the source code may contact the corresponding author, Hao Wei at UO302709@uniovi.es. During the preparation of this work the author(s) used Open AI's ChatGPT in order to enhance language quality and assist with proofreading. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Declarations

Conflicts of Interest All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter discussed in this manuscript.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

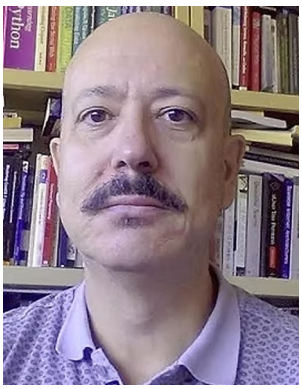
1. Sarker, I.H.: Machine learning: Algorithms, real-world applications and research directions. SN computer science **2**(3), 160 (2021)
2. Lwakatare, L.E., Raj, A., Crnkovic, I., Bosch, J., Olsson, H.H.: Large-scale machine learning systems in real-world industrial settings: A review of challenges and solutions. Inf. Softw. Technol. **127**, 106368 (2020)
3. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., Dennison, D.: Hidden technical debt in machine learning systems. In: Cortes, C., Lawrence, N., Lee, D., Sugiyama, M., Garnett, R. (eds.), Advances in Neural Information Processing Systems, vol. 28. Curran Associates, Inc (2015)
4. Zhengxin, F., Yi, Y., Jingyu, Z., Liu Yue, M., Qinghua, Y.L., Xiwei, X., Jeff, W., Chen, W., Shuai, Z. Shiping, C.: Mlops spanning whole machine learning life cycle: A survey. https://doi.org/10.48550/arXiv.2304.07296 (2023)
5. Kreuzberger, D., Kühl, N., Hirschl, S.: Machine learning operations (mlops): Overview, definition, and architecture. IEEE Access **11**, 31866–31879 (2023)
6. S &P Global Market Intelligency. Multicloud in the mainstream. techreport, S &P Global Market Intelligency, February (2023)
7. Ayas, H.M., Leitner, P., Hebig, R.: An empirical study of the systemic and technical migration towards microservices. Empir. Softw. Eng. **28**(4), 85 (2023)
8. Fragiadakis, G., Liagkou, V., Filiopoulou, E., Fragkakis, D., Michalakelis, C., Nikolaidou, M.: Cloud services cost comparison: a clustering analysis framework. Computing **105**(10), 2061–2088 (2023)
9. Laaber, C., Scheuner, J., Leitner, P.: Software microbenchmarking in the cloud. how bad is it really? Empir. Softw. Eng. **24**(4), 2469–2508 (2019)
10. Giray, G.: A software engineering perspective on engineering machine learning systems: State of the art and challenges. J. Syst. Softw. **180**,(2021)
11. Paleyes, A., Urma, R.-G., Lawrence, N.D.: Challenges in deploying machine learning: A survey of case studies. ACM Comput. Surv. **55**(6), 1–29 (2022)
12. Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., Zimmermann, T.: Software engineering for machine learning: A case study. In 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pages 291–300 (2019)
13. Moe, N.B., Šmite, D., Paasivaara, M., Lassenius, C.: Finding the sweet spot for organizational control and team autonomy in large-scale agile software development. Empir. Softw. Eng. **26**(5), 101 (2021)
14. Shridhar, A., Nadig, D.: Heuristic-based resource allocation for cloud-native machine learning workloads. In 2022 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS), pages 415–418 (2022)
15. Chiang, M.-C., Zhang, L.-W., Chou, Y.-M., Chou, J.: Dynamic resource management for machine learning pipeline workloads. SN Computer Science **4**(5), 665 (2023)
16. Chiang, M.-C., Chou, J.: Dynamoml: Dynamic resource management operators for machine learning workloads. In Proceedings of the 11th International Conference on Cloud Computing and Services Science-Volume 1: CLOSER, pages 122–132. INSTICC, SciTePress (2021)
17. Miñón, R., Arcaya, J.D., Torre-Bastida, A.I., Hartlieb, P.: Pangea: An mlops tool for automatically generating infrastructure and deploying analytic pipelines in edge, fog and cloud layers. Sensors **22**(12), 4425 (2022)

18. Zhang, Y., Hua, W., Zhou, Z., Suh, G.E., Delimitrou, C.: Sinan: Ml-based and qos-aware resource management for cloud microservices. In Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '21, page 167-181, New York, NY, USA. Association for Computing Machinery (2021)
19. Gholipour, N., Arianyan, E., Buyya, R.: A novel energy-aware resource management technique using joint vm and container consolidation approach for green computing in cloud data centers. Simul. Model. Pract. Theory **104**, 102127 (2020)
20. Zhu, Q.-H., Tang, H., Huang, J.-J., Hou, Y.: Task scheduling for multi-cloud computing subject to security and reliability constraints. IEEE/CAA Journal of Automatica Sinica **8**(4), 848–865 (2021)
21. Mangalampalli, S., Karri, G.R., Ratnamani, M.V., Mohanty, S.N., Jabr, B.A., Ali, Y.A., Ali, S., Abdullaeva, B.S.: Efficient deep reinforcement learning based task scheduler in multi cloud environment. Sci. Rep. **14**(1), 21850 (2024)
22. Ghasemi, A., Haghighat, A.T., Keshavarzi, A.: Enhancing virtual machine placement efficiency in cloud data centers: a hybrid approach using multi-objective reinforcement learning and clustering strategies. Computing **106**(9), 2897–2922 (2024)
23. Li, Z., Cheng, Q., Hsieh, K., Dang, Y., Huang, P., Singh, P., Yang, X., L, Qin, Wu, Y., Levy, S., Chintalapati, M.: Gandalf: An intelligent, End-To-End analytics service for safe deployment in Large-Scale cloud infrastructure. In 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), pages 389–402, Santa Clara, CA, February. USENIX Association (2020)
24. Khan, S., Gani, A., Wahab, A.W.A., Guizani, M., Khan, M.K.: Topology Discovery in Software Defined Networks: Threats, Taxonomy, and State-of-the-Art. IEEE Communications Surveys & Tutorials **19**(1), 303–324 (2017)
25. Hwang, J., Liu, G., Zeng, S., Wu, F.Y., Wood, T.: Topology Discovery and Service Classification for Distributed-Aware Clouds. In 2014 IEEE International Conference on Cloud Engineering, pages 385–390. IEEE (2014)
26. Sangpetch, A., Kim, H.S.: VDEP: VM Dependency Discovery in Multi-tier Cloud Applications. In 2015 IEEE 8th International Conference on Cloud Computing, pages 694–701. IEEE (2015)
27. Lutellier, T., Chollak, D., Garciam, J., Tan, L., Derek, R., Medvidovic, N., Kroeger, R.: Comparing software architecture recovery techniques using accurate dependencies. 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering **2**, 69–78 (2015)
28. Zhang, Y., Xu, Z., Liu, C., Chen, H., Sun, J., Qiu, D., Liu, Y.: Software architecture recovery with information fusion. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, page 1535–1547, New York, NY, USA. Association for Computing Machinery (2023)
29. Link, D., Behnamghader, P., Moazeni, R., Boehm, B.: Recover and relax: Concern-oriented software architecture recovery for systems development and maintenance. In 2019 IEEE/ACM International Conference on Software and System Processes (ICSSP), pages 64–73, May (2019)
30. Link, D., Behnamghader, P., Moazeni, R., Boehm, B.: The value of software architecture recovery for maintenance. In Proceedings of the 12th Innovations in Software Engineering Conference (Formerly Known as India Software Engineering Conference), ISEC '19, New York, NY, USA. Association for Computing Machinery (2019)
31. Ullmann, G.C., Guéhéneuc, Y.-G., Petrillo, F., Anquetil, N., Politowski, C.: Visualising game engine subsystem coupling patterns. In Paolo Ciancarini, Angelo Di Iorio, Helmut Hlavacs, and Francesco Poggi, editors, Entertainment Computing– ICEC, pages 263–274, Singapore, 2023. Springer Nature Singapore (2023)
32. Lutellier, T., Chollak, D., Garcia, J., Tan, L., Rayside, D., Medvidović, N., Kroeger, R.: Measuring the impact of code dependencies on software architecture recovery techniques. IEEE Trans. Software Eng. **44**(2), 159–181 (2018)
33. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition (2015)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Hao Wei is a researcher with academic training in software engineering and electronic science. He holds a Master's degree in Software Engineering from Beijing Jiaotong University, and a Bachelor's degree in Electronic Science from Beijing Normal University. He conducted research at the Universidad Politécnica de Madrid and Universidad de Oviedo, focusing on cloud computing, service development, and access control. His work has led to several peer-reviewed publications on topics such as hybrid and multi-cloud, medical image processing, and cloud security frameworks. In addition to his academic pursuits, he has accumulated extensive professional experience in cloud infrastructure and software engineering through roles in both industry and research institutions.

Xabiel García Pañeda has a Ph.D. degree (2004) from the University of Oviedo and is currently a Professor at the same university. His main research areas are the Internet of Things and edge computer architectures. He has been working on these topics and others such as HCI and learning technologies to design advanced driving assistant systems for the last ten years.



Joaquín Salvachúa Rodríguez is a Professor at the Universidad Politécnica de Madrid (UPM), specializing in telematics and cyber-physical systems. His recent research focuses on FIWARE-based digital twin architectures, IoT interoperability, and semantic metadata for AI transparency. He also has a strong foundation in formal methods and multimedia real-time protocols (co-author of an RFC) and teaches cloud infrastructure, big data, blockchain, and DLT technologies. An active contributor to Gaia-X Hub Spain and a participant in European forums on data and AI innovation.